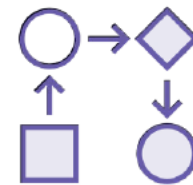




等待线程的结束

北京理工大学计算机学院
金旭亮

等待另一个线程的结束



A线程正在运行，希望插入一个B线程，要求B先执行完毕，A才继续运行，可以使用join方法达到这个目的：

```
public final void join() throws InterruptedException
```

join的含义是：将某一线程加入，成为另一个线程的流程的一部分。



join方法的作用，相当于执行该方法的线程和线程调度器说：“我得先暂停一下，等到另一个线程运行结束之后我才能继续干活。”

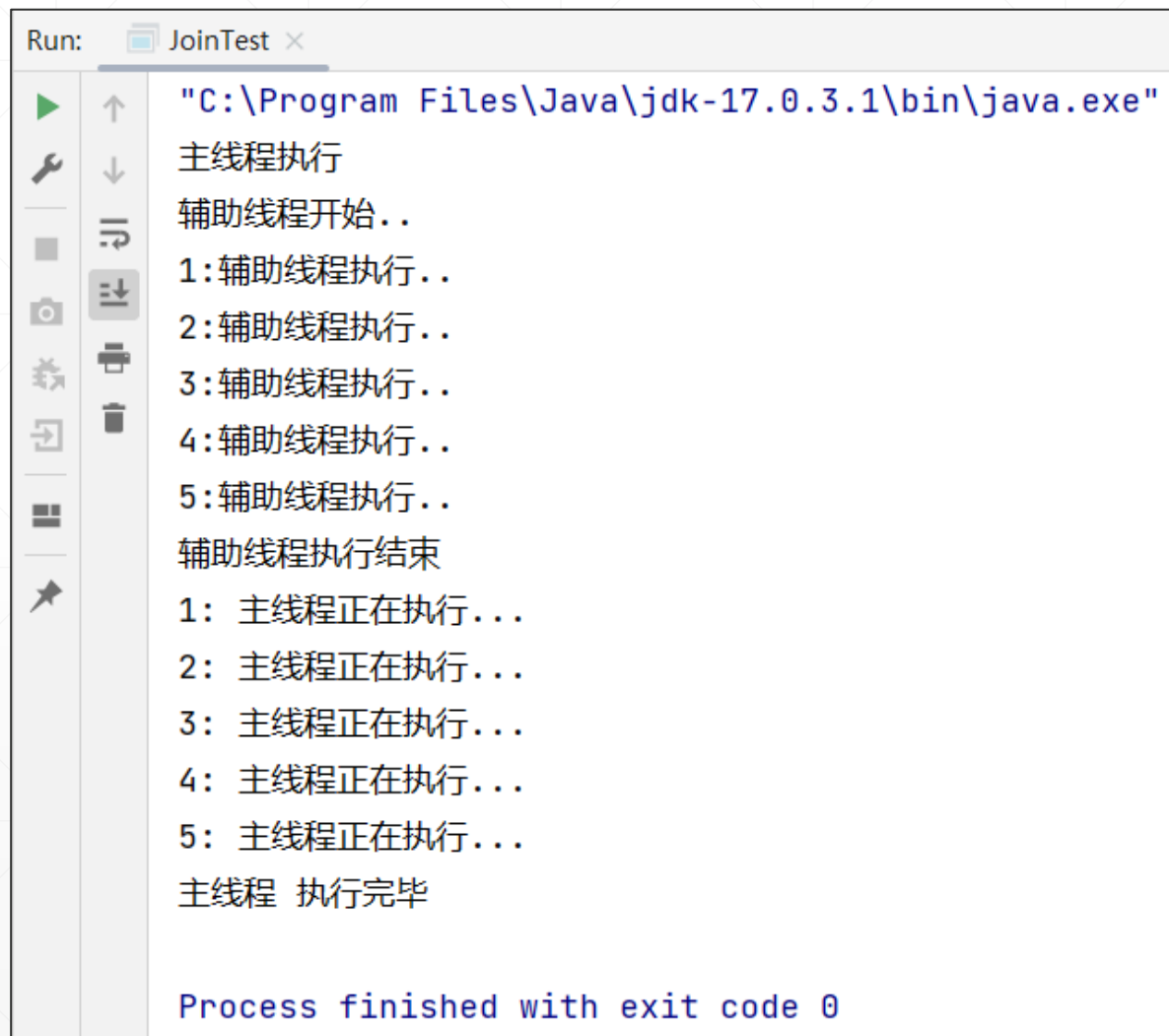
Join示例源码分析

```
public class JoinTest {  
    public static void main(String[] args)  
        throws InterruptedException {  
        System.out.println("主线程执行");  
        Thread otherThread = new Thread(() -> {  
            try {  
                System.out.println("辅助线程开始..");  
                for (int i = 1; i <= 5; i++) {  
                    Thread.sleep(1000);  
                    System.out.println(i + ":辅助线程执行..");  
                }  
                System.out.println("辅助线程执行结束");  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        });  
        otherThread.start(); //启动线程
```

主线程启动辅助线程，然后等待它执行完毕之后再运行。

```
        try {  
            //辅助线程加入主线程  
            otherThread.join();  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        for (int i = 1; i <= 5; i++) {  
            Thread.sleep(500);  
            System.out.println(i + ": 主线程正在执行...");  
        }  
  
        System.out.println("主线程 执行完毕");  
    }  
}
```

Join示例截图



```
Run: JoinTest x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
主线程执行
辅助线程开始..
1:辅助线程执行..
2:辅助线程执行..
3:辅助线程执行..
4:辅助线程执行..
5:辅助线程执行..
辅助线程执行结束
1: 主线程正在执行...
2: 主线程正在执行...
3: 主线程正在执行...
4: 主线程正在执行...
5: 主线程正在执行...
主线程 执行完毕

Process finished with exit code 0
```



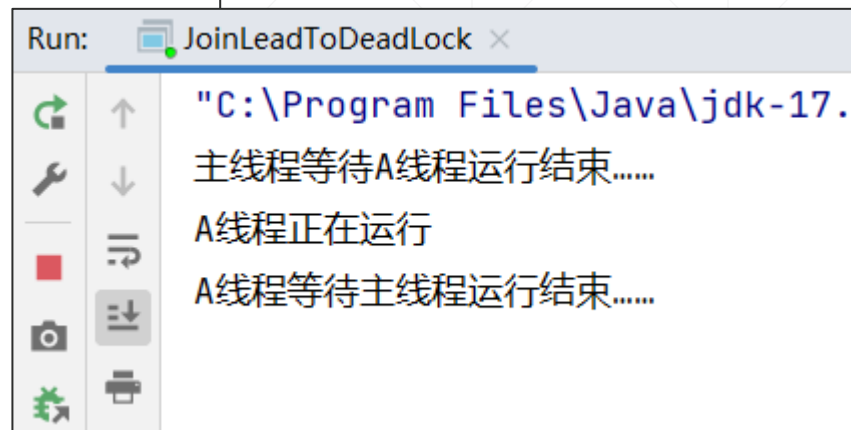
如果不使用join方法，会导致两个线程交错执行，可以注释掉join那句代码，自己试一试。

JoinTest.java

不恰当的join，导致死锁.....



```
public class JoinLeadToDeadLock {  
    public static void main(String[] args) throws InterruptedException {  
        var mainThread : Thread = Thread.currentThread();  
        Thread aThread = new Thread(() -> {  
            System.out.println("A线程正在运行");  
            try {  
                System.out.println("A线程等待主线程运行结束.....");  
                mainThread.join();  
            } catch (InterruptedException e) {  
                throw new RuntimeException(e);  
            }  
        });  
        System.out.println("主线程等待A线程运行结束.....");  
        aThread.start();  
        aThread.join();  
        System.out.println("程序运行结束");  
    }  
}
```



死锁



所谓“死锁”，是多线程程序可能出现的一种特殊状态，在这种状态下，多个线程都没有运行结束，但由于相互等待，而谁也无法继续运行，导致程序“僵”在一个地方，动弹不得.....

有期限的等待



`join()` 方法有一个重载版本，不是无限期地等待被调用的线程对象执行完毕，而是限定最长等待多久——过时不候！

```
void join(long milliseconds)
```



例如，如果在 `thread1` 中调用了 `thread2.join(1000)`，那么，`thread1` 线程暂停执行，直到遇到以下两个条件之一：

1. `thread2` 结束执行。
2. 1000 ms 的等待时间结束。


```
public class JoinWithTimeoutNotLeadToDeadLock {  
    public static void main(String[] args) throws InterruptedException {  
        var mainThread : Thread = Thread.currentThread();  
        Thread aThread = new Thread(() -> {  
            System.out.println("A线程正在运行");  
            try {  
                System.out.println("A线程等待主线程运行结束,最长等待2秒.....");  
                mainThread.join( millis: 2000);  
            } catch (InterruptedException e) {  
                throw new RuntimeException(e);  
            }  
            System.out.println("A线程运行结束");  
        });  
        System.out.println("主线程等待A线程运行结束.....");  
        aThread.start();  
        aThread.join();  
        System.out.println("程序运行结束");  
    }  
}
```

设定了超时的join, 可以避免死锁的出现。

JoinWithTimeoutNotLeadToDeadLock.java

```
Run: JoinWithTimeoutNotLeadToDeadLock x  
"C:\Program Files\Java\jdk-17.0.3.1  
主线程等待A线程运行结束.....  
A线程正在运行  
A线程等待主线程运行结束,最长等待2秒.....  
A线程运行结束  
程序运行结束  
Process finished with exit code 0
```